

11,0
EU ENSINO ESSA DISCIPLINA TEM UM TEMPO,
E COM CERTEZA ESSA PROVA FOI A MELHOR QUE EU JÁ CORRI! PARABÉNS!

Campus de Quixadá

Universidade Federal do Ceará

Segunda Avaliação Parcial

Linguagens de Programação

Prof. Lucas Ismaily

1º Semestre de 2026

Aluno: [Guilherme Wallyberto Feres] Matrícula: [58210]

Informações importantes:

- A prova vale 11 pontos;
- Caso não saiba responder uma questão, se você escrever “não sei” ganhará 10% da questão. Por exemplo, se uma questão vale 2 pontos, você receberá 0,2 décimos;
- **Prova individual.** Sejam honestos com vocês e comigo, por favor. Se for detectado qualquer tipo de fraude, os envolvidos receberão nota zero. Note: os envolvidos receberão zero, não importa se você foi a origem ou o destino, ambos receberão nota zero.

Questões:

1. (3,0 pontos) Em algumas linguagens de programação, o tipo de uma variável não é determinado permanentemente no momento de sua declaração. Em vez disso, a variável pode receber valores de diferentes tipos durante a execução do programa, e a vinculação entre a variável e seu tipo ocorre dinamicamente. Esse comportamento é comum em linguagens como Python e JavaScript e está frequentemente associado ao uso de variáveis heap-dinâmicas, cujas células de memória são alocadas e desalocadas durante a execução.

Com base nesse contexto, explique o que é vinculação dinâmica de tipos e descreva sua relação com as variáveis heap-dinâmicas. Em sua resposta, apresente pelo menos uma vantagem e uma desvantagem decorrentes dessa escolha de projeto de linguagem.

2. (2,0 pontos) As linguagens de programação utilizam diferentes critérios para determinar se dois tipos de dados devem ser considerados equivalentes. Na equivalência por nome, dois tipos são equivalentes quando possuem a mesma declaração ou o mesmo nome de origem. Na equivalência por estrutura, a comparação é realizada considerando a organização interna dos tipos, como os campos, seus tipos e a ordem em que aparecem. Analise essas duas formas de equivalência e explique qual é a principal vantagem da equivalência de tipos por nome. Em seguida, apresente e justifique uma desvantagem da equivalência de tipos por estrutura, considerando situações em que tipos conceitualmente distintos podem possuir representações internas semelhantes.

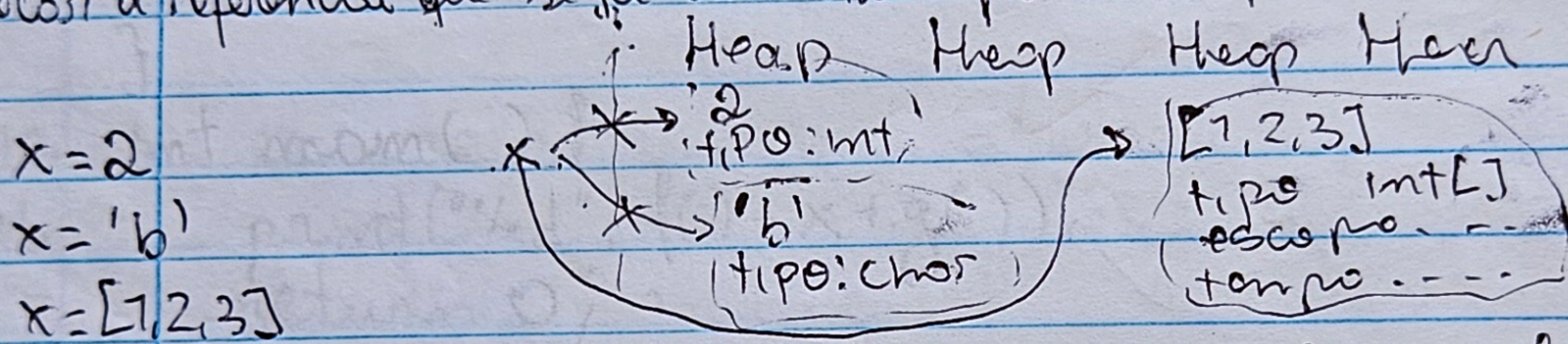
3. (3,0 pontos) Considere uma linguagem que permite que durante uma expressão possa existir uma função que modifique variáveis externas ao seu escopo local; e considere também que utiliza operadores lógicos como 'AND' e 'OR'. Durante a avaliação de uma expressão lógica, a linguagem pode interromper o processamento assim que o resultado final puder ser determinado. Entretanto, essa interrupção pode fazer com que determinadas funções presentes na expressão não sejam executadas, afetando eventuais alterações realizadas por elas no estado do programa.

Com base nessa situação, defina os conceitos de efeito colateral funcional e avaliação de curto-circuito. Depois, explique como a combinação desses dois mecanismos pode fazer com que o comportamento de um programa dependa da ordem de avaliação das expressões.

4. (3,0 pontos) Na implementação de chamadas de subprogramas, uma linguagem pode optar por copiar o valor de um argumento para o parâmetro formal ou transmitir apenas um caminho de acesso para o objeto original. Na passagem por referência, o parâmetro formal funciona como um apelido para a variável utilizada como argumento, permitindo que o subprograma acesse e modifique diretamente o objeto existente no ambiente do chamador.

Considerando os aspectos de desempenho, uso de memória, segurança e legibilidade do código, explique as principais vantagens e desvantagens da passagem de parâmetros por referência. Em sua resposta, discuta também os problemas que podem surgir quando dois ou mais parâmetros formais fazem referência à mesma posição de memória.

3. Vinculação dinâmica de tipos é aquela que o tipo só pode ser identificado em run-time, diferentemente de vinculação estática de tipos que é identificada em tempo de compilação. Com a vinculação dinâmica, o variável também pode trocar seu tipo durante a execução do programa. Uma vantagem é a flexibilidade de poder trocar o tipo ~~durante a execução~~ durante a execução. Outra vantagem é a menor eficiência em relação às estáticas, pois devem ser alocados objetos em tempo de execução. Agora comentando sobre sua relação com variáveis-heap dinâmicas, que são aqueles que ficam no heap e podem crescer, diminuir e a vontade. No troco de tipo (dinâmico) não é necessário criar outro variável com tipo diferente, é só trocar a referência que se ja de outro tipo no heap ex Java Script



4. Passagem por referência no geral é melhor em desempenho e uso de memória pois se precisar passar um objeto grande por copia no parametro real ~~para~~ para o formal, o programa precisaria literalmente copiar ~~para~~ o objeto para a ~~memória do processo~~ o escopo no chamado do filho do instância daquela função, por exemplo um array multidimensional muito grande, ~~precisaria ocupar~~ vai ocupar muito memória "desnecessariamente", enquanto no programa por referência o que se passa é o endereço do objeto, sem precisar copiar ele inteiro.

→ e modifica diretamente o objeto

O que se diz a segurança, pode haver algumas confusões usando referenciamento e possivelmente prejudicando a segurança pois o código não funcionaria como esperado

2. A principal vantagem na verificação por nome, é a facilidade de implementação: essa verificação entre duas variáveis, por óbvio, não olham para o nome do tipo, ocasionando em uma verificação bem mais eficiente e rápida que por estrutura. Uma das vantagens na verificação por estrutura, é a maior dificuldade de implementação, e chegoem mais lentos, como antes foi mencionado, então desvantagem é que é possível fazer operações com tipos de ~~seg~~ significados diferentes mas com estruturas semelhantes, por exemplo em uma calculadora, seria possível uma adição de $'a' + 3$, já que int e char são estruturalmente semelhantes mas que provavelmente não faz sentido em certas situações.

4. Continuação: pois estaria mudando valores de uma variável dentro do função, exemplo:

```
void f(int *x) {
    *x = 4;
}

int main() {
    int a = 10;
    printf("%d", a); // 10
    f(&a);
    printf("%d", a); // 40
```

Erroneamente problema pode atribuir no legibilidade, ~~mas~~ entre problema de legibilidade seria o código, considere o código:

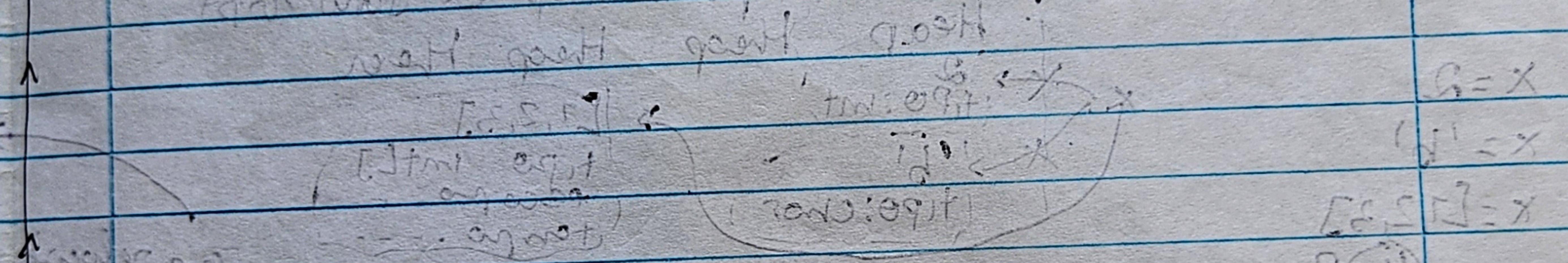
```
void f(int *x, int *y) {
    *x = 0;
    *y = 1;
}

int main() {
    int a = 10;
    f(&a, &a);
```

se levarmos este tempo ao erro operacional de ineficiência

Nesse caso onde os dois ~~problemas~~ parâmetros formais referem-se ao mesmo endereço na memória, gera o problema já comentado de aliasing, onde duas variáveis referenciam o mesmo bloco de código, criando um quilo, e prejudicando a legibilidade, e as vezes com comportamento inesperado.

→ Por isso algumas linguagens tem sobrecarga de operadores para se / e $\&$, com isso $\&\&$ e $\&\&\&$ para anular a contagem - enquanto se / e $\&$ para não -

[illegible]

The first part of the paper is devoted to a discussion of the
 various methods of determining the rate of reaction. It is
 found that the rate of reaction is proportional to the
 concentration of the reactants. This is in agreement with
 the law of mass action. The rate of reaction is also
 affected by the presence of a catalyst. The rate of
 reaction is increased by the presence of a catalyst. The
 rate of reaction is decreased by the presence of an inhibitor.